RAPPORT TECHNIQUE

Déploiement d'un Web Application Firewall avec NGINX & NAXSI

Architecture, détection, journalisation et première évaluation des performances

Périmètre : laboratoire de sécurité web basé sur DVWA, NGINX et le module NAXSI.

Environnement principal : Ubuntu Server 22.04.5 LTS · NGINX 1.18 · NAXSI 1.7 · Kali Linux

Date du rapport : 11 octobre 2026

Résumé exécutif

Ce projet a consisté à placer un WAF devant l'application de laboratoire DVWA en utilisant NGINX comme reverse proxy et NAXSI comme moteur de détection et de blocage. Les travaux ont porté sur le déploiement du module, la configuration des règles, la validation de scénarios contrôlés, l'observation des journaux et une première comparaison des temps de réponse.

Les essais consignés montrent des blocages HTTP 403 pour plusieurs requêtes de test SQLi, XSS, traversal/LFI, de type RFI et d'évasion. Les journaux NAXSI ont permis de relier certaines requêtes aux règles déclenchées et aux scores associés. Une campagne de 30 requêtes par chemin a donné un temps moyen affiché de 7,1 ms en accès direct au backend et de 10,3 ms via le WAF. Ces mesures sont préliminaires : toutes les réponses étaient des redirections HTTP 302 et le test ne permet pas d'isoler le coût propre de NAXSI.

Conclusion synthétique : le laboratoire démontre le fonctionnement de base du filtrage et de la journalisation sur les scénarios testés, mais ne constitue ni une certification de sécurité ni une validation pour la production.

Sommaire

- 1. Contexte et objectifs
- 2. Architecture et environnement
- 3. Déploiement et configuration
- 4. Politique de filtrage NAXSI
- 5. Campagne de tests de sécurité
- 6. Journalisation et analyse
- 7. Évaluation préliminaire des performances
- 8. Limites et recommandations
- 9. Conclusion
- Annexe A — Commandes de diagnostic

1. Contexte et objectifs

Une application web exposée peut recevoir des requêtes malveillantes visant notamment l'injection SQL, le XSS, l'accès à des chemins non autorisés ou l'exploitation de mécanismes de traitement des entrées. Un WAF constitue une couche de défense supplémentaire : il inspecte les requêtes HTTP avant qu'elles atteignent l'application.

1.1 Objectif général

Déployer et évaluer un WAF fondé sur NGINX et NAXSI devant DVWA, dans un environnement de laboratoire contrôlé.

1.2 Objectifs spécifiques

- Installer et charger le module NAXSI et ses règles principales.
- Configurer NGINX en reverse proxy vers le backend DVWA.
- Définir des seuils de score pour plusieurs catégories de détection.
- Exécuter des scénarios contrôlés et vérifier les codes HTTP retournés.
- Analyser les journaux d'accès et les événements NAXSI.
- Comparer les temps de réponse du backend direct et du chemin passant par le WAF.

2. Architecture et environnement

2.1 Composants

Composant	Configuration observée	Fonction
Client de test	Kali Linux — 192.168.195.128	Émission des requêtes de test
Serveur WAF	Ubuntu Server 22.04.5 LTS — 192.168.195.146	Hébergement du proxy et du WAF
NGINX	1.18.0-6ubuntu14.21, port 80	Point d'entrée HTTP / reverse proxy
NAXSI	Module dynamique 1.7	Inspection des requêtes HTTP
Règles principales	/etc/nginx/naxsi/naxsi_core.rules	Règles de détection
Configuration du site	/etc/nginx/sites-available/waf	Proxy et politique de blocage
Backend DVWA	Apache sur 127.0.0.1:3001	Application de test
Journal des accès	/var/log/nginx/access.log	URI et statuts HTTP
Journal d'erreurs	/var/log/nginx/error.log	Erreurs et événements NAXSI

2.2 Flux de traitement

```
Client Kali (192.168.195.128)
          | HTTP / port 80
          v
Serveur WAF (192.168.195.146)
  NGINX + module NAXSI
          |
          +-- Requête bloquée ------> HTTP 403 au client
          |
          +-- Requête autorisée ---> DVWA / Apache
                                     127.0.0.1:3001
          |
          +-- Journaux : access.log et error.log
```

Le backend est configuré sur l'interface de boucle locale. Le client de test passe normalement par le port 80 du WAF ; l'accès direct au port backend sert ici à la comparaison de performance depuis le serveur.

3. Déploiement et configuration

Le module NAXSI 1.7 a été construit et installé sous forme de module dynamique. Le fichier du module est /usr/lib/nginx/modules/ngx_http_naxsi_module.so. Les règles principales ont été placées dans /etc/nginx/naxsi/naxsi_core.rules.

La configuration du site est gérée dans /etc/nginx/sites-available/waf. Le proxy transmet les requêtes autorisées à http://127.0.0.1:3001. Le test de syntaxe NGINX a été indiqué comme réussi dans le contexte du projet.

3.1 Comportement attendu

1. Les requêtes reçues par le port 80 sont inspectées par NAXSI.
2. Les requêtes qui atteignent les seuils configurés sont bloquées.
3. Les requêtes autorisées sont transmises au backend.
4. Les événements et statuts HTTP sont analysés dans les journaux.

4. Politique de filtrage NAXSI

Catégorie	Seuil de blocage configuré
SQL	$SQL >= 8
XSS	$XSS >= 8
RFI	$RFI >= 8
Traversal	$TRAVERSAL >= 5
EVADE	$EVADE >= 4

Les scores sont alimentés par les règles déclenchées. Lorsque le seuil d'une catégorie est atteint, la politique de blocage peut provoquer une réponse HTTP 403. Une règle déclenchée n'indique pas à elle seule qu'une attaque a réussi ; elle doit être interprétée avec l'URI, la variable concernée, le statut HTTP et le contexte du test.

4.1 URI interne de refus

```
location = /RequestDenied {
    internal;
    return 403;
}
```

La directive internal réserve cette URI à un usage interne dans NGINX. La configuration active doit être vérifiée avant réutilisation dans un autre environnement.

5. Campagne de tests de sécurité

Les scénarios suivants ont été exécutés dans le laboratoire via le WAF. Les résultats sont repris des observations de console et des journaux partagés pendant le projet.

Scénario	Observation enregistrée	Interprétation
SQL Injection (SQLi)	Score $SQL=22 ; règles 1009 et 1013 ; config=block	La requête de test a été bloquée selon la politique observée.
XSS réfléchi	HTTP 403 ; $XSS=8 ; règle 1010	Le seuil XSS configuré a été atteint.
Traversal / test LFI	HTTP 403 ; $TRAVERSAL observé jusqu'à 16 ; règle 1200	La requête de test a été bloquée.
Requête de type RFI	HTTP 403 ; $RFI=8 ; règle 1100	Le seuil RFI configuré a été atteint.
Test EVADE avec %25U	HTTP 403 ; $EVADE=4 ; règle 1401	Le score a atteint le seuil EVADE configuré.

Chemin doublement encodé	Règle 1315 ; événement classé dans les scores XSS	La catégorie dépend de la règle déclenchée et du jeu de règles chargé.

5.1 Analyse du test EVADE

Le test effectué avec la valeur %25U a produit une réponse HTTP 403 et un événement associé à la règle 1401, avec un score $EVADE de 4. Le seuil configuré étant $EVADE >= 4, le score observé atteint ce seuil. EVADE désigne ici une catégorie de score liée à des motifs d'évasion reconnus par les règles ; ce n'est pas le nom d'une attaque unique.

5.2 Requêtes légitimes de contrôle

Trois requêtes considérées comme bénignes ont été vérifiées ; aucun événement de blocage correspondant n'a été identifié dans l'échantillon examiné. Cette observation ponctuelle ne permet pas de conclure à l'absence générale de faux positifs.

6. Journalisation et analyse

Deux fichiers principaux ont été utilisés : access.log pour les traces d'accès, URI et codes HTTP ; error.log pour les erreurs NGINX et événements NAXSI, notamment ceux contenant NAXSI_FMT.

```
# Dernières lignes du journal des accès
sudo tail -n 50 /var/log/nginx/access.log

# Événements NAXSI
sudo grep 'NAXSI_FMT' /var/log/nginx/error.log

# Occurrences NAXSI présentes dans le journal
sudo grep -c 'NAXSI_FMT' /var/log/nginx/error.log

# Événements consignés comme blocages
sudo grep 'NAXSI_FMT' /var/log/nginx/error.log | grep -c 'config=block'

# Vérifier la syntaxe NGINX
sudo nginx -t
```

Pendant l'observation de la phase de journalisation, grep -c NAXSI_FMT a renvoyé 39. Ce nombre est un comptage d'occurrences dans le journal consulté à ce moment-là ; il ne doit pas être présenté comme 39 attaques uniques.

Les codes HTTP doivent être corrélés avec les événements NAXSI. Un HTTP 302 était également observé lors de l'accès à DVWA sans authentification ; ce code n'est donc pas nécessairement un échec du WAF.

7. Évaluation préliminaire des performances

Une campagne de 30 requêtes séquentielles par chemin a été exécutée depuis le serveur Ubuntu. Le temps total de chaque requête a été relevé avec curl, puis agrégé avec awk.

Indicateur	Backend direct	Via NGINX + NAXSI
Nombre de requêtes	30	30
Temps moyen	0,0071 s (7,1 ms)	0,0103 s (10,3 ms)
Temps minimum	0,0021 s (2,1 ms)	0,0030 s (3,0 ms)
Temps maximum	0,0679 s (67,9 ms)	0,1188 s (118,8 ms)
Codes HTTP	302 pour les 30 requêtes	302 pour les 30 requêtes

7.1 Calculs et interprétation

Écart moyen à partir des valeurs affichées : 10,3 ms − 7,1 ms = environ 3,2 ms.

Surcoût relatif estimé : ((10,3 − 7,1) / 7,1) × 100 ≈ 45,1 %. Le pourcentage est indicatif, car il repose sur des moyennes arrondies et des temps variables.

Toutes les réponses étaient des redirections HTTP 302, cohérentes avec la redirection de DVWA vers la connexion. L'écart compare deux chemins complets et ne mesure pas le coût propre de NAXSI isolément. Il peut inclure le traitement NGINX, le reverse proxy, le backend et les variations d'exécution. Le test est séquentiel et ne représente pas une charge concurrente.

7.2 Améliorations proposées

- Effectuer plusieurs séries et appliquer une courte phase de chauffe.
- Mesurer une ressource identique retournant HTTP 200 dans les deux chemins.
- Calculer la médiane et les percentiles, notamment P95.
- Surveiller CPU, mémoire et charge système pendant les mesures.
- Conserver les données brutes pour rendre les calculs reproductibles.

8. Limites et recommandations de sécurité

- DVWA est volontairement vulnérable : le laboratoire doit rester isolé et ne pas être exposé sur Internet.
- N'effectuer des tests que sur des systèmes détenus ou explicitement autorisés.
- Valider les seuils NAXSI avec des requêtes légitimes représentatives afin de limiter les faux positifs.
- Corréler chaque statut HTTP avec les événements NAXSI et les journaux du backend.
- Vérifier la configuration active après toute modification et exécuter sudo nginx -t avant rechargement.
- Maintenir le système, NGINX et les règles de sécurité à jour.
- Ne pas considérer les scénarios testés comme une preuve que toutes les variantes d'attaque sont bloquées.
- Compléter les tests fonctionnels et de performance avant toute réflexion sur un déploiement en production.

9. Conclusion

Le projet a permis de déployer un chemin d'accès web protégé par NGINX et NAXSI devant DVWA, d'appliquer des seuils de score et d'observer plusieurs blocages sur des scénarios contrôlés. Les journaux NGINX ont fourni des éléments pour corréler les requêtes, les règles déclenchées et les codes HTTP.

La première campagne de performance a mesuré 7,1 ms en moyenne pour l'accès direct au backend et 10,3 ms via le WAF. L'écart d'environ 3,2 ms constitue une référence initiale à consolider par des essais complémentaires sur une ressource HTTP 200 et par des statistiques plus robustes.

Bilan : architecture de laboratoire déployée, tests de détection/blocage réalisés, journalisation analysée et première campagne de performance documentée. Le système n'a pas fait l'objet d'une certification de sécurité.

Annexe A — Commandes de diagnostic

```
# Vérifier l'état du service
sudo systemctl status nginx
```

```
# Vérifier la configuration avant rechargement
sudo nginx -t

# Recharger NGINX après validation
sudo systemctl reload nginx

# Suivre le journal d'erreurs
sudo tail -f /var/log/nginx/error.log

# Tester le backend depuis le serveur WAF
curl -sS -o /dev/null -w 'Backend | HTTP %{http_code} | %{time_total}s\n' http://127.0.0.1:3001/

# Tester le chemin passant par le WAF
curl -sS -o /dev/null -w 'WAF | HTTP %{http_code} | %{time_total}s\n' http://127.0.0.1/
```